

Sketchpad

A Little Shape-Drawing Kit

Object-Oriented Programming — Standalone Lab

Marks: 8 **Language:** Java (JDK 17) **Approach:** Design a small class family from a description
No unit tests. A Canvas and a Main are given to you — you write the shapes, the failures, and the gallery. Submit your
.java files and a short design note.

The job

Build the *shape* half of a tiny console drawing tool, **Sketchpad**. The Canvas that actually prints the picture, and a Main that runs everything, are given — you do not touch them. You write three things: a **family of shape classes**, a **family of exceptions** a shape throws when it cannot be drawn, and a **gallery** that draws a pile of mixed shapes and, when one cannot be drawn, skips it, says why, and keeps going. There is no test file: run Main, read the picture and the gallery's report.

What you build

1. **A shape hierarchy** — a top type Shape, and concrete kinds Circle, Rectangle, and Square. Every shape can:
 - report its **area** (circle πr^2 , rectangle $w \times h$, square s^2);
 - give a one-line **description** of itself;
 - **draw itself** on a Canvas — but only after checking it fits; if it does not fit it throws (see below) and paints *nothing*.

A shape built with a nonsensical size (negative radius, side, width, or height ≤ 0) must be rejected *in its constructor* with IllegalArgumentException.
2. **An exception family** for “this shape cannot be drawn right now”: two **checked** kinds — ShapeTooLargeException (bigger than the canvas can hold) and ShapeTooSmallException (would not cover even one cell) — which the gallery must be able to catch *together, in one place*. How you arrange them so that becomes possible is part of the design (see decision 3).
3. **A gallery** that holds a pile of shapes, draws them all in one loop, catches each failure and continues, then reports how many it drew, how many it skipped and why, and shows the canvas.

▷ Notice

The Canvas is provided. You **do not compute pixels** — you call its ready-made primitives (paint a circle of radius r , paint a rectangle $w \times h$, ask how much room it has, show the picture). Your shapes' only job when drawing is to *check they fit*, then hand the actual painting to the Canvas. Exact method names and signatures are in the Canvas source you are given.

The three decisions you are marked on

Passing eyeballs on the printed picture is easy; the marks are in these three design choices, which are also where people go wrong.

1. Which parent is abstract, which is concrete?

You have *two* parent classes, and they are not the same kind of parent.

- Shape sits at the top and stands for “any shape at all.” It has no form of its own — there is no way to draw a plain “shape” or give it an area formula. So Shape is **abstract**: `new Shape(...)` must not compile.
- Rectangle is *itself* a real, drawable shape **and** the parent of Square (a square is a rectangle with equal sides). A class can be both a working shape and a parent, so Rectangle stays **concrete**.

Where people trip: making Rectangle abstract too “to be safe,” or making Shape concrete and leaving its area returning 0. Ask of each parent: *does a bare instance of this make sense?* If yes → concrete; if no → abstract.

2. When do you override a method the parent already wrote?

Only when the parent’s version is *wrong for this child* — not out of habit.

- Square **must override** Rectangle’s “change the width” and “change the height” methods, so that changing one side changes both and the square stays square. The parent’s versions compile and run — but they let a square stop being square, so they are wrong *here*.
- Square **must not override** area: the rectangle’s $w \times h$ already gives s^2 when the sides are equal. Circle *does* override the default description (it has a radius to mention); Rectangle keeps the default.

Where people trip: overriding area in Square for no reason, or overriding a method just to call super and add nothing. If your override doesn’t change behaviour, delete it.

3. Which exception, and where is it thrown?

Two different failures, two different tools — keep them apart.

- A shape with an *impossible size* is a bug in the caller: throw `IllegalArgumentException` (unchecked) *from the constructor*, so a broken shape never exists.
- A *valid* shape that will not fit the canvas is an expected outcome the gallery must survive: throw a **checked** `ShapeTooLargeException` or `ShapeTooSmallException` *from the draw method*, and throw it **before painting a single mark** so the canvas is left untouched.

Where people trip: using the same exception for both, printing an error instead of throwing, or painting first and checking after (a half-drawn shape). Constructor = unchecked, reject bad data; draw = checked, refuse a good shape the canvas can’t hold.

Think ahead about the catch. The gallery has to stop *every* drawing failure in a *single* spot in its loop — and it should still work the day a third kind of failure (a shape that runs off the edge, say) is added. If `ShapeTooLargeException` and `ShapeTooSmallException` are two *unrelated* types, count how many catch blocks that one loop needs today, and how many more it grows every time a new failure kind appears. What relationship between these exceptions would let *one* catch hold all of them at once — today’s two and tomorrow’s third — without the loop being reopened each time? Decide that before you write either exception.

Question Your Viva Will Ask

(abstract or concrete) Why is `new Shape()` nonsense but `new Rectangle(...)` fine, when both are parents?

(overriding) You overrode Square’s side-changers but not its area. Why is one override necessary and the other a mistake?

(dispatch) Your gallery’s loop asks every shape “draw yourself” by one name, yet each paints differently and may throw differently. Who picks the right code for each shape, and when — as

you wrote the loop, or as it runs?

The gallery must survive failure

The gallery holds circles, rectangles, and squares together as plain Shapes — its loop never asks which kind a shape is. For each shape it tries to draw, catches a “cannot be drawn” exception if one is thrown, records the shape and the reason, and moves to the next. One over-sized shape must never stop the shapes behind it. Afterwards it reports how many were drawn, how many skipped and why, and shows the canvas.

Submit

Your .java files (not Canvas or Main), plus **four sentences**: which class you made abstract and why; which parent you left concrete and why; one method you overrode though the parent implemented it, and why; one method you deliberately did *not* override, and why.

Marking Rubric

#	Criterion	Marks
1	The top of the family is <i>abstract</i> — a bare shape cannot be constructed — and states what every shape can do (area, paint, draw) without being drawable itself	2.0
2	The rectangle is a <i>concrete</i> shape and the parent of a square; the square <i>overrides</i> the side-changers to stay square, while <i>inheriting</i> area and description unchanged	2.0
3	Failures use exceptions correctly: a nonsensical shape is rejected at construction with <code>IllegalArgumentException</code> (unchecked); “cannot be drawn” is a small <i>checked</i> family (one parent, two kinds), thrown before any mark is painted	2.0
4	The gallery renders a mixed pile in one branch-free sweep, <i>catches</i> each refusal and continues, and reports drawn / skipped / reasons; a shape that does not fit leaves the canvas untouched	1.5
5	Polymorphism: one pile holds every kind of shape, and one variable can point at any shape, with no “which kind am I” ladder anywhere	0.5
Total		8

▷ Notice

There are no tests to hide behind here. Criteria 1, 2, and 3 are judged by *reading your code* and by the viva: that a bare shape is impossible to construct, that a square overrides exactly the methods it must and no more, and that failure travels only as a thrown exception. A design that reaches the right picture through one class with a kind string, or that overrides a square’s area needlessly, or that never throws, misses the point of the lab and will score accordingly.

Ground rules

- Java 17. Use double for sizes and areas, int for counts.
- Use a `java.util.ArrayList` for the gallery's pile (and for the list of skipped shapes in your report). The container is not the point; *what* you can put in it — any kind of shape, under one common type — is.
- Canvas and Main are provided. **Do not modify them**, and do not try to draw the pixels yourself — call the Canvas primitives. There is no test file; you check your work by running Main and reading the picture and the report.
- Say every relationship aloud before you commit to it: a square *is a* rectangle; a gallery *has a* pile of shapes. And say every failure aloud: is it a *broken shape* (reject at construction, unchecked) or a *good shape the canvas refuses* (throw a checked exception, caught by the gallery)?
- Partial credit is awarded; submit whatever compiles, plus your four-sentence design note.